Last Updated: 12/19/2025 01:17:11

Open API 2.0 Developer Documentation

Instruction

Through Open API service, Buyers and Seller can integrate third-party ERP systems or self-developed systems with GIGAB2B to access relevant data and develop custom applications.

Currently, Buyers can apply for Open API connection by accessing: Buyer Central → APIs → Connection Options tab → Open API. Once the application is successfully submitted and proceeds to active status, Buyers can view the details in the Active Connections tab to obtain their Client ID and Client Secret. For Sellers requesting Open API integration, they are required to contact the Marketplace Online Customer Service for activation and obtain their Client ID and Client Secret.

Connection Process:
1.Buyers: Apply for Open API connection via GIGAB2B and obtain their API Key.
2.Sellers: Request Open API connection by contacting Marketplace Online Customer Service. (Note: Currently only On-Site Sellers are supported.)
3.Configure common parameters according to this documentation and set up signatures using the provided Signature Rules.
4.Test and deploy the integration. If Buyers require GIGAB2B 's assistance during testing, contact Marketplace Online Customer Service to connect with technical support.

Version Notes

The rules mentioned in this documentation apply to all Open API 2.0 interfaces. Developers are advised to migrate to this version as soon as possible, as it offers more robust features, standardized error messages, and richer feedback. The legacy version (Open API 1.0) will soon be deprecated and will no longer be accessible.

API Keys obtained by Buyers before the release of the new version (prior to August 28, 2025) on the GIGAB2B can be used to access all Open API 2.0 interfaces, with few exceptions for those who applied offline. If your existing API Key fails to call Open API 2.0 interfaces, please reapply for Open API connection on GIGAB2B as early as possible by accessing: Buyer Central → APIs → Connection Options → Open API to obtain a new API Key.

URLs for Each Environment

Test Environment URL: https://openapi-sandbox.gigab2b.com
Production Environment URL: https://openapi.gigab2b.com
Note: The above URLs are for the OpenAPI 2.0 version interfaces, not for the 1.0 version. Do not confuse the two. For the 1.0 version, refer to the GIGA Open API Document in the Help Center.

Signature Rules

Prerequisites: Buyers must first obtain an API Key from GIGAB2B, which consists of the Client ID and Client Secret provided on the Connection Details page. Sellers need to contact Marketplace Customer Service for Open API connection and their Client ID and Client Secret. Note that Client ID and Client Secret must be in the production environment.
1.Construct String 1: Concatenate the API Key, API path, timestamp (from the request header), and nonce (random value) using "&" as the delimiter. Example: Client ID&API path&timestamp&nonce
2.Construct the Secret Key: Concatenate the Client ID, Client Secret, and the nonce (from the request header) using "&" as the delimiter. Example: Client ID&Client Secret&nonce
3.Use the secret key to perform an HMAC-SHA256 encryption on String 1, and then convert the result into a hexadecimal string to obtain Ciphertext 1
4.Sign Value: Encode Ciphertext 1 in Base64 to obtain the final signature (sign), which is passed as an input parameter.

Code example:

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.util.Base64;

/**
 * api sign tool
 */
public class ApiSignUtil {
    public static final String NAME = "HmacSHA256";

    /**
     * encrypt
     *
     * @param message (input parameters)
     * @param secretKey (secret key)
     * @return  (return encrypted value)
     */
    public static String encrypt(String message, String secretKey) {
        try {
            // create HMAC-SHA256  sample
            Mac hmacSHA256 = Mac.getInstance(NAME);
            // create secret key specifications
            SecretKeySpec spec = new SecretKeySpec(secretKey.getBytes(StandardCharsets.UTF_8), NAME);
            //  initialize HmacSHA256  sample
            hmacSHA256.init(spec);
            byte[] bytes = hmacSHA256.doFinal(message.getBytes(StandardCharsets.UTF_8));
            // convert to hexadecimal
            StringBuilder hexStr = new StringBuilder();
            for (byte b : bytes) {
                hexStr.append(String.format("%02x", b));
            }
            return hexStr.toString();
        } catch (NoSuchAlgorithmException | InvalidKeyException e) {
            throw new RuntimeException(e);
        }
    }

    /**
     * access sign
     * @param clientId clientId
     * @param timestamp
     * @param nonce (10 digit random string)
     * @param uri (request url)
     * @param clientSecret clientSecret
     * @return api (return API sign)
     */
    public String getApiSign(String clientId, Long timestamp, String nonce, String uri, String clientSecret) {
        String msg = clientId + "&" + uri + "&" + timestamp + "&" + nonce;
        String key = clientId + "&" + clientSecret + "&" + nonce;
        byte[] result = encrypt(msg, key).getBytes(StandardCharsets.UTF_8);
        return Base64.getEncoder().encodeToString(result);
    }

    public static void main(String[] args) {
        String clientId = "123456789";
        String clientSecret = "123456789";
        String uri = "/api/v1/test";
        String nonce = "1234567890";
        Long timestamp = System.currentTimeMillis();
        String apiSign = new ApiSignUtil().getApiSign(clientId, timestamp, nonce, uri, clientSecret);
        System.out.println(apiSign);
    }
}
```

Common Parameters

The GIGA Open API input parameters consist of two parts: common parameters and request parameters. Request parameters can be either query parameters or request body parameters. The common parameters are as follows:

Parameter Name	Data Type	Required ?	Description	Default Value
Content-Type	String	YES	Input parameter data type specification. The default value is application/json.	application/json
client-id	String	YES	Client ID. Part of the API Key applied by Buyers on GIGAB2B for Open API connection.	
timestamp	String	YES	The millisecond value converted, representing the number of seconds elapsed from January 1, 1970 (UTC) to current time. Only requests within 20 minutes are processed.	
nonce	String	YES	10-digit random value	
sign	String	YES	Signature	

Query Parameters

For GET requests, append parameters in key-value pairs after the API endpoint URL, using ? as the connector, separated by '&'. For example: GET /api-b2b-v1/product/skus?orderId=123

```
GET /api/items?item_id=123&sort=desc
```

Request body parameters

POST, PUT, and similar request methods often carry more business parameters and involve more complex data structures. Example:

```
Content-Type: application/json
{
  "skus": [
     "W59463028"
  ]
}
```

Response Parameters

GIGAB2B has established a similar response parameter structure for each interface. Except for a few interfaces that may include additional parameters, the interface structure follows the table below:

Parameter Name	Data Type	Required ?	Description	Default Value
success	bool	YES	Determining request success	
code	string	YES	Status Code: Returns 200 for success, or the corresponding error code for failures.	
data	object[]	NO	Information about the request response.	
requestId	string	YES	A unique identifier to help client-side track and handle exceptions.	
msg	string	YES	Error message: Returns "success" on success, or an error description on failure.	
subMsg	string	NO	Error details scenario description: returns when the response fails and includes a secondary error.	
recommend	string	NO	Error Diagnostics URL: Client can directly copy the URL to navigate to the error diagnostics page (refer to the "Error Codes" section in the API documentation) for quick issue resolution.	

Request Rate Limit

To ensure API performance and stability, request rate limit is enforced. Refer to the respective API documentation for specific rate limits per interface.